

MARAN PMS — Módulo de Facturación

PROMPT: Facturación Electrónica — Modo Ficticio + Preparado para ARCA

CONTEXTO

El hotel emite facturas a huéspedes y empresas. Actualmente el sistema genera comprobantes internos del folio. Este prompt implementa la emisión de **facturas fiscales reales** (A y B), con dos modos:

- **Modo FICTICIO** (default): genera facturas con CAE simulado, para usar durante pruebas y validación del flujo. Los PDFs se ven idénticos a los reales.
- **Modo ARCA** (activable por configuración): conecta con la API real de AFIP/ARCA para obtener el CAE verdadero.

El toggle entre modos se controla desde **Configuración → Facturación → Modo producción ARCA**.

PARTE 1 — CONFIGURACIÓN

Tabla en `shared/schema.ts`

```
export const billingConfig = pgTable("billing_config", {
  id: serial("id").primaryKey(),
  // Modo de operación
  modoArca: boolean("modo_arca").default(false),
  // false = ficticio, true = ARCA real

  // Datos del emisor (Maran SA)
  cuit: text("cuit").default("33-68110008-9"),
  razonSocial: text("razon_social").default("MARAN S.A."),
  domicilioComercial: text("domicilio_comercial").default("Alameda de la Federaci
  localidad: text("localidad").default("Paraná"),
  provincia: text("provincia").default("Entre Ríos"),
```

```

cp: text("cp").default("3100"),
condicionIva: text("condicion_iva").default("Responsable Inscripto"),
inicioActividades: text("inicio_actividades").default("01/01/2000"),

// Punto de venta registrado en ARCA
puntoVenta: integer("punto_venta").default(1),
// Tipo de punto de venta: "online" (web services) | "electronico"
tipoPuntoVenta: text("tipo_punto_venta").default("online"),

// Credenciales ARCA (solo se usan en modo_arca = true)
// IMPORTANTE: guardar encriptado en producción
arcaCert: text("arca_cert"),          // certificado digital X.509
arcaKey: text("arca_key"),            // clave privada
arcaCuit: text("arca_cuit"),          // CUIT del representante autorizado

// Logo para el PDF de factura
logoUrl: text("logo_url"),

updatedAt: timestamp("updated_at").defaultNow(),
});

```

Tabla de facturas emitidas

```

export const salesInvoices = pgTable("sales_invoices", {
  id: serial("id").primaryKey(),

  // Tipo de comprobante
  tipoComprobante: text("tipo_comprobante").notNull(),
  // "FA" = Factura A | "FB" = Factura B | "FC" = Factura C
  // "NCA" = Nota Crédito A | "NCB" = Nota Crédito B

  // Numeración
  puntoVenta: integer("punto_venta").notNull(),
  numero: integer("numero").notNull(),
  // La combinación (tipoComprobante, puntoVenta, numero) es única

  // Fechas
  fechaEmision: date("fecha_emision").notNull(),
  fechaVtoPago: date("fecha_vto_pago"),

  // Datos del cliente
  // Para FA: empresa con CUIT
  // Para FB: consumidor final o persona física
  clienteRazonSocial: text("cliente_razon_social").notNull(),
  clienteCuit: text("cliente_cuit"),          // requerido para FA
  clienteDni: text("cliente_dni"),           // alternativa para FB

```

```

clienteCondicionIva: text("cliente_condicion_iva").notNull(),
// "Responsable Inscripto" | "Consumidor Final" | "Monotributista" | "Exento"
clienteDomicilio: text("cliente_domicilio"),

// Montos
montoNeto: numeric("monto_netto", { precision: 14, scale: 2 }).notNull(),
montoIva21: numeric("monto_iva21", { precision: 14, scale: 2 }).default("0"),
montoIva105: numeric("monto_iva105", { precision: 14, scale: 2 }).default("0"),
montoExento: numeric("monto_exento", { precision: 14, scale: 2 }).default("0"),
montoNoGravado: numeric("monto_no_gravado", { precision: 14, scale: 2 }).default("0"),
montoTotal: numeric("monto_total", { precision: 14, scale: 2 }).notNull(),

// CAE (Código de Autorización Electrónica)
cae: text("cae"),
caeFechaVto: date("cae_fecha_vto"),
// En modo ficticio: CAE simulado con formato válido (14 dígitos)
// En modo ARCA: CAE real obtenido de AFIP

// Modo en que se generó
modoFicticio: boolean("modo_ficticio").default(true),

// Estado
estado: text("estado").default("emitida"),
// "emitida" | "anulada" (las facturas electrónicas no se anulan, se emite NC)

// Vínculos
reservaId: integer("reserva_id"), // FK a reservations
folioId: integer("folio_id"), // FK al folio/cuenta del huésped
notaCreditoId: integer("nota_credito_id"), // si es NC, FK a la factura origin

// Concepto para el PDF
concepto: text("concepto").default("2"),
// "1" = Productos | "2" = Servicios | "3" = Productos y Servicios

// Detalle de los ítems facturados
items: jsonb("items"),
// Array de: { descripcion, cantidad, precioUnitario, alicuotaIva, subtotal }

operador: text("operador"),
createdAt: timestamp("created_at").defaultNow(),
});

// Contador de próximo número por tipo y punto de venta
export const invoiceCounters = pgTable("invoice_counters", {
  id: serial("id").primaryKey(),
  tipoComprobante: text("tipo_comprobante").notNull(),
  puntoVenta: integer("punto_venta").notNull(),
});

```

```
    ultimoNumero: integer("ultimo_numero").default(0),  
    // unique constraint: (tipoComprobante, puntoVenta)  
  });
```

PARTE 2 — SERVICIO DE FACTURACIÓN

server/billing/invoiceService.ts

```
import { getBillingConfig } from "../billingConfig";  
import { generateFakeCAE } from "../fakeArca";  
import { callARCA } from "../arcaClient"; // se implementa en Parte 4  
  
export async function emitirFactura(data: NewInvoiceData): Promise<SalesInvoice>  
  const config = await getBillingConfig();  
  
  // 1. Obtener próximo número  
  const numero = await getNextInvoiceNumber(data.tipoComprobante, config.puntoVen  
  
  // 2. Calcular montos  
  const montos = calcularMontos(data);  
  
  // 3. Obtener CAE  
  let cae: string;  
  let caeFechaVto: Date;  
  let modoFicticio: boolean;  
  
  if (config.modosArca) {  
    // Modo real: llamar a ARCA  
    const resultado = await callARCA({  
      tipo: data.tipoComprobante,  
      puntoVenta: config.puntoVenta,  
      numero,  
      ...montos,  
      cliente: data.cliente,  
    });  
    cae = resultado.cae;  
    caeFechaVto = resultado.caeFechaVto;  
    modoFicticio = false;  
  } else {  
    // Modo ficticio: generar CAE simulado  
    const fake = generateFakeCAE();  
    cae = fake.cae;  
    caeFechaVto = fake.vencimiento;
```

```

        modoFicticio = true;
    }

    // 4. Guardar en base de datos
    const [factura] = await db.insert(salesInvoices).values({
        tipoComprobante: data.tipoComprobante,
        puntoVenta: config.puntoVenta,
        numero,
        fechaEmision: new Date(),
        clienteRazonSocial: data.cliente.razonSocial,
        clienteCuit: data.cliente.cuit,
        clienteCondicionIva: data.cliente.condicionIva,
        clienteDomicilio: data.cliente.domicilio,
        ...montos,
        cae,
        caeFechaVto,
        modoFicticio,
        reservaId: data.reservaId,
        folioId: data.folioId,
        items: data.items,
        operador: data.operador,
    }).returning();

    return factura;
}

// Generador de CAE ficticio - formato idéntico al real (14 dígitos numéricos)
function generateFakeCAE(): { cae: string; vencimiento: Date } {
    const cae = Array.from({ length: 14 }, () => Math.floor(Math.random() * 10)).join('');
    const vencimiento = new Date();
    vencimiento.setDate(vencimiento.getDate() + 10); // vence en 10 días
    return { cae, vencimiento };
}

// Cálculo de montos según tipo de comprobante
function calcularMontos(data: NewInvoiceData) {
    let montoNeto = 0;
    let montoIva21 = 0;
    let montoIva105 = 0;
    let montoExento = 0;
    let montoNoGravado = 0;

    for (const item of data.items) {
        switch (item.alicuotaIva) {
            case "21":
                montoNeto += item.subtotalNeto;
                montoIva21 += item.subtotalNeto * 0.21;
            case "105":
                montoNeto += item.subtotalNeto;
                montoIva105 += item.subtotalNeto * 0.105;
            case "Exento":
                montoNeto += item.subtotalNeto;
            case "NoGravado":
                montoNoGravado += item.subtotalNeto;
        }
    }
}

```

```

        break;
    case "10.5":
        montoNeto += item.subtotalNeto;
        montoIva105 += item.subtotalNeto * 0.105;
        break;
    case "exento":
        montoExento += item.subtotal;
        break;
    case "no_gravado":
        montoNoGravado += item.subtotal;
        break;
    }
}

// Para Factura B: el precio ya incluye IVA (el cliente no es Resp. Inscripto)
// Los montos internos se calculan igual pero el total es el precio con IVA inc
const montoTotal = montoNeto + montoIva21 + montoIva105 + montoExento + montoNo

return { montoNeto, montoIva21, montoIva105, montoExento, montoNoGravado, monto
}

```

PARTE 3 — ENDPOINTS

```

POST    /api/billing/invoices
→ Emitir nueva factura
→ body: { tipoComprobante, cliente, items[], reservaId?, folioId? }
→ Retorna: factura completa con CAE y URL del PDF

GET     /api/billing/invoices?desde=&hasta=&tipo=&clienteCuit=
→ Listar facturas emitidas con filtros

GET     /api/billing/invoices/:id
→ Detalle de una factura

GET     /api/billing/invoices/:id/pdf
→ Descargar el PDF de la factura

POST    /api/billing/invoices/:id/nota-credito
→ Emitir nota de crédito que anula/ajusta una factura
→ body: { motivo, items[] }
→ Genera automáticamente la NC del mismo tipo (NCA si era FA, NCB si era FB)

GET     /api/billing/config
→ Obtener configuración de facturación

```

```
PATCH  /api/billing/config
→ Actualizar configuración (incluye toggle modoArca)

GET    /api/billing/next-number?tipo=FA
→ Consultar próximo número disponible
```

PARTE 4 — CLIENTE ARCA (preparado pero desactivado)

server/billing/arcaClient.ts

```
// Este módulo está preparado para conectar con ARCA cuando se active el modo rea
// Documentación: https://www.afip.gob.ar/fe/documentos/

import { getBillingConfig } from "../billingConfig";

// Códigos de tipo de comprobante ARCA
const TIPOS_COMPROBANTE: Record<string, number> = {
  "FA": 1,    // Factura A
  "FB": 6,    // Factura B
  "FC": 11,   // Factura C
  "NCA": 3,   // Nota de Crédito A
  "NCB": 8,   // Nota de Crédito B
};

// Códigos de condición IVA del receptor
const CONDICION_IVA: Record<string, number> = {
  "Responsable Inscripto": 1,
  "Monotributista": 6,
  "Exento": 4,
  "Consumidor Final": 5,
};

// Alícuotas IVA ARCA
const ALICUOTAS_IVA: Record<string, number> = {
  "21": 5,    // 21%
  "10.5": 4,  // 10.5%
  "exento": 3, // Exento
  "no_gravado": 1, // No gravado
};

export async function callARCA(invoiceData: ARCAInvoiceData): Promise<ARCAResult> {
  const config = await getBillingConfig();
```

```

if (!config.modoArca) {
  throw new Error("ARCA está desactivado. Activar en Configuración → Facturació
}

if (!config.arcaCert || !config.arcaKey) {
  throw new Error("Faltan credenciales ARCA. Configurar certificado y clave pri
}

// 1. Obtener token de autorización (WSAA)
const token = await getWSAAToken(config.arcaCert, config.arcaKey);

// 2. Llamar al Web Service de Facturación Electrónica (WSFE)
const wsfeUrl = "https://servicios1.afip.gov.ar/wsfev1/service.asmx"; // produc
// const wsfeUrl = "https://wshomo.afip.gov.ar/wsfev1/service.asmx"; // homolo

const soapBody = buildWSFERequest({
  token,
  cuit: config.cuit,
  tipo: TIPOS_COMPROBANTE[invoiceData.tipo],
  puntoVenta: invoiceData.puntoVenta,
  numero: invoiceData.numero,
  fechaEmision: format(new Date(), "yyyyMMdd"),
  montoTotal: invoiceData.montoTotal,
  montoNeto: invoiceData.montoNeto,
  montoIva: invoiceData.montoIva21 + invoiceData.montoIva105,
  montoExento: invoiceData.montoExento,
  montoNoGravado: invoiceData.montoNoGravado,
  alicuotasIva: buildAlicuotasIva(invoiceData),
  cuitReceptor: invoiceData.cliente.cuit,
  condicionIvaReceptor: CONDICION_IVA[invoiceData.cliente.condicionIva],
  concepto: 2, // Servicios
});

const response = await fetch(wsfeUrl, {
  method: "POST",
  headers: { "Content-Type": "text/xml" },
  body: soapBody,
});

const xmlResult = await response.text();
const parsed = parseWSFEResponse(xmlResult);

if (parsed.resultado !== "A") {
  throw new Error(`ARCA rechazó la factura: ${parsed.errores.join(", ")}`);
}

```



```

    return {
      cae: parsed.cae,
      caeFechaVto: parse(parsed.caeFechaVto, "yyyyMMdd", new Date()),
    };
  }

  // Funciones auxiliares (implementar según documentación ARCA)
  async function getWSAAToken(cert: string, key: string): Promise<WSAAToken> {
    // Implementar autenticación WSAA con certificado X.509
    // Ver: https://www.afip.gob.ar/ws/WSAA/
    throw new Error("Implementar WSAA");
  }

  function buildWSFERequest(data: any): string {
    // Construir XML SOAP para FECAESolicitar
    throw new Error("Implementar WSFE request");
  }

  function parseWSFEResponse(xml: string): any {
    // Parsear respuesta XML
    throw new Error("Implementar WSFE response parser");
  }

```

Nota: Las funciones `getWSAAToken`, `buildWSFERequest` y `parseWSFEResponse` están definidas pero lanzan error intencionalmente hasta que se activen. En modo ficticio nunca se llaman.

PARTE 5 — PDF DE FACTURA

`server/billing/invoicePdf.ts`

El PDF debe cumplir con el **diseño oficial AFIP** para facturas electrónicas.

[LOGO MARAN]	
MARAN S.A.	FACTURA
CUIT: 33-68110008-9	Código 001 (A) / 006 (B)
Responsable Inscripto	N° 0001-00000001
Alameda de la Fed. 698	
Paraná, Entre Ríos (3100)	Fecha: DD/MM/YYYY
Inicio actividades: 01/01/2000	

PERÍODO FACTURADO: DD/MM/YYYY al DD/MM/YYYY
FECHA DE VTO. PARA EL PAGO: DD/MM/YYYY

SR/ES: [Razón Social Cliente]
DOMICILIO: [Domicilio]
CUIT/CUIL: XX-XXXXXXXX-X
CONDICIÓN DE IVA: Responsable Inscripto

Cant	Descripción	P/U	Subtotal
3	Alojamiento habitación doble - 11/03 al 14/03	\$X.XXX	\$X.XXX
1	Desayuno buffet	\$XXX	\$XXX

Subtotal:

\$X.XXX,XX

IVA 21%:

\$XXX,XX

TOTAL:

\$XX.XXX,XX

SON PESOS [IMPORTE EN LETRAS]

CAE N°: 12345678901234Vto. CAE: DD/MM/YYYY

[CÓDIGO DE BARRAS] [CODIGO QR AFIP]

⚠ [Si modoFicticio = true]:

** DOCUMENTO NO VÁLIDO COMO COMPROBANTE FISCAL **

** GENERADO EN MODO DE PRUEBA **

Código QR: AFIP exige un QR en las facturas electrónicas desde 2022. URL del QR:

`https://www.afip.gob.ar/fe/qr/?p=[BASE64_del_JSON]`

Donde el JSON contiene: ver, fecha, cuit, ptoVta, tipoCmp, nroCmp, importe, moneda, ctz, tipoDocRec, nroDocRec, tipoCodAut, codAut.

En modo ficticio, generar el QR igual pero con datos simulados (el QR existe pero apunta a datos no registrados en AFIP).

PARTE 6 — FRONTEND

Ruta: /admin/facturacion

 Facturación

[+ Nueva Factura]

[Este mes] [Mes anterior] [Rango] [Buscar por CUIT]

Tipo	Número	Cliente	Total	Estado
FA	0001-00000001	EMPRESA SA	\$12.100	✓
FB	0006-00000045	Juan Pérez	\$8.470	✓
NCA	0003-00000001	EMPRESA SA	-\$12.100	✓
...				

[Si modoFicticio]

⚠ MODO PRUEBA — Las facturas no tienen validez fiscal. Activar ARCA en Configuración

NewInvoiceDialog — Nueva Factura

Tipo de comprobante:

[FA - Factura A] [FB - Factura B]

FA: para Responsables Inscriptos (empresas con CUIT)

FB: para Consumidores Finales, Monotributistas, Exentos

— Datos del cliente —

CUIT/DNI: [input] [Buscar en padrón AFIP ↗]
(en modo ficticio: solo autocomplete local)

Razón Social: [input]

Condición IVA: [Select automático según tipo de comprobante]

Domicilio: [input]

— Ítems a facturar —

[Cargar desde folio de reserva] o [Agregar ítem manual]

Descripción	Cant	P. Neto	Subtotal
Alojamiento hab. 301...	3	\$5.000	\$15.000
[+ Agregar ítem]			

Alicuota IVA: [Select: 21% | 10.5% | Exento | No Gravado]

Totales	
Neto gravado 21%:	\$15.000,00
IVA 21%:	\$3.150,00
Total:	\$18.150,00

[Emitir Factura]

Al emitir → spinner “Obteniendo CAE...” → muestra el PDF generado → opción de descargar o imprimir.

Integración con Folio de Recepción

En la pantalla de checkout del folio, agregar botón **“Emitir Factura”** que pre-carga el dialog con los ítems del folio ya cargados. Al emitir la factura, el folio queda marcado como “facturado”.

PARTE 7 — CONFIGURACIÓN ARCA

Ruta: /admin/facturacion/configuracion

Configuración de Facturación	
DATOS DEL EMISOR	
CUIT:	33-68110008-9
Razón Social:	MARAN S.A.
Domicilio:	Alameda de la Federación 698
Punto de Venta:	0001
MODO DE OPERACIÓN	
☐ Modo Prueba (ficticio) ☒ Modo ARCA (producción)	
[Solo visible si modo ARCA]	
Certificado ARCA (.crt):	[Subir archivo]
Clave privada (.key):	[Subir archivo]
[Testear conexión con ARCA]	
☒ Conexión exitosa / ☒ Error: [mensaje]	
[Guardar configuración]	

NOTAS

1. **Numeración:** El próximo número de cada tipo se incrementa en la tabla `invoiceCounters` . Es transaccional — usar `SELECT FOR UPDATE` para evitar duplicados si dos facturas se emiten al mismo tiempo.
2. **Factura A vs B:**
 - FA: el precio en los ítems es **sin IVA** (neto). Se suma el IVA al total.
 - FB: el precio en los ítems es **con IVA incluido**. El IVA se discrimina pero está incluido.
 - El formulario adapta las etiquetas automáticamente según el tipo seleccionado.
3. **Validación del CUIT:** en modo ficticio, validar solo el formato (11 dígitos + dígito verificador). En modo ARCA, opcionalmente consultar el padrón AFIP.
4. **Facturas de grupos:** cuando se factura una empresa que ocupó varias habitaciones, agregar un ítem por habitación. El total es la suma de todas.
5. **Nota de Crédito:** solo se emite para corregir o anular una factura ya emitida. Debe referenciar el CAE de la factura original. El sistema no permite “anular” directamente — siempre se genera la NC correspondiente.